AI-Powered Personal Memory System for Web Content Recall

CSD 415 - PROJECT PHASE 1

MDL22CS086	22CSB20	GAYATHRI K BINOY
MDL22CS104	22CSB27	JESLIA ROSE
MDL22CS124	22CSB34	MEGHA B
MDL22CS184	22CSB58	STHUTHI SAMBATH

B. Tech. Computer Science & Engineering

Department of Computer Engineering
Model Engineering College, Ernakulam
Thrikkakara, Kochi - 682021
Phone: 91.484.2575370

http://www.mec.ac.in

hodcs@mec.ac.in

October 2025

Model Engineering College, Ernakulam

Dept. of Computer Engineering

C E R T I F I C A T E

This is to certify that, this report titled ***"AI-Powered Personal Memory System for Web Content Recall "*** is a bonafide record of the work done by:

MDL22CS086	22CSB20	**GAYATHRI K BINOY**
MDL22CS104	22CSB27	**JESLIA ROSE**
MDL22CS124	22CSB34	**MEGHA B**
MDL22CS184	22CSB58	**STHUTHI SAMBATH**

Seventh Semester B. Tech. Computer Science & Engineering students, for the course work in **CSD 415 - Project Phase 1** which is the Main Project Work, under our guidance and supervision, in partial fulfillment of the requirements for the award of the degree, B. Tech. Computer Science and Engineering of **APJ Abdul Kalam University**.

Guide

Mrs. Amrutha M
Assistant Professor
Computer Engineering

Coordinator

Dr. Priya S
Professor
Computer Engineering

Head of the Department

Dr. Binu V. P
Professor
Computer Engineering

October, 2025

Acknowledgements

We are profoundly grateful to **Mrs. Amrutha M** for expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

We would like to express deepest appreciation towards **Dr. Jayachandran E S**, Principal, Govt. Model Engineering College, Kochi, **Dr. Binu V P**, Head of Department of Computer Engineering and **Dr. Priya S**, Project Coordinator whose invaluable guidance supported us in completing this project.

At last we must express our sincere heartfelt gratitude to all the staff of Computer Engineering Department who helped us directly or indirectly during this course of work.

22CSB20	GAYATHRI K BINOY
22CSB27	JESLIA ROSE
22CSB34	MEGHA B
22CSB58	STHUTHI SAMBATH

Abstract

In today's digital first world, individuals engage with a large volume of information daily through articles, PDFs, and videos. Despite this constant consumption, much of the knowledge absorbed during browsing sessions is quickly forgotten, often buried deep in browser history or lost entirely. Existing tools like bookmarks or note-taking apps offer manual and fragmented solutions that fail to address the need for contextual and intelligent recall.

This project introduces an AI-powered personal memory system that captures, understands, and stores the meaningful content a user interacts with during their web browsing. Through a browser extension, users can choose when to start and stop recording their activity. During this time, the system captures content from webpages, YouTube videos via transcripts, and browser-viewed PDFs. This content is then processed using natural language processing techniques such as summarization, topic extraction, and semantic embedding, and is stored alongside metadata like source URLs and timestamps.

Users can later access a web interface that offers a clean, organized dashboard of their browsing activity. A conversational AI assistant enables users to ask questions about their past interactions, allowing them to retrieve summaries, revisit useful links, and surface related images they chose to save. The system prioritizes user privacy and control, allowing them to exclude specific tabs or delete recorded data at any time. By combining intelligent content capture with semantic recall, this project offers a seamless and personalized way to transform passive browsing into meaningful, retrievable knowledge, bridging the gap between what users consume and what they remember.

Contents

1 Introduction **10**

1.1 Proposed Project 10

1.1.1 Background 10

1.1.2 Objective of the Project 10

2 Literature Survey **12**

2.1 Introduction 12

2.2 Literature Survey Table 12

2.3 Detailed Literature Survey 14

2.3.1 A Pragmatic Approach to Web-Summarizer System[1] 14

2.3.2 Advancements in the Efficacy of Flan-T5 for Abstractive Text Summarization: A Multi-Dataset Evaluation Using ROUGE and BERT Scores[2] 16

2.3.3 AI-Powered Transcript Summarization and Question Answering for YouTube and Wikipedia[3] 18

2.3.4 A Brief Survey of Vector Databases[4] 19

2.3.5 Enhancing Transformer-Based Semantic Search by Integrating Human-Crafted Knowledge into Sentence Embeddings[5] 20

2.3.6 Automatic Webpage Content Extraction Based on Structural and Semantic Features[6] 22

2.3.7 Sentence Meaning Similarity Detector Using FAISS[7] 23

2.3.8 Research on Sentence Embeddings for Text Matching through Multiview Interactive Features[8] 25

2.3.9 AI-based Feature Extraction and Cosine Similarity for Automation of Student Learning Assessment [9] 26

2.4 Future Scope from Literature 28

3 Problem Statement and Solution 29

3.1 Problem Statement 29

3.1.1 Problem Statement 29

3.1.2 Proposed Solution 29

4 Software Requirements Specification (SRS) 31

4.1 Introduction 31

4.1.1 Purpose 31

4.1.2 Document Conventions 31

4.1.3 Intended Audience and Reading Suggestions 32

4.1.4 Project Scope 32

4.1.5 Overview of Developer's Responsibilities 32

4.2 Overall Description 33

4.2.1 Product Perspective 33

4.2.2 Product Functions 33

4.2.3 User Classes and Characteristics 33

4.2.4 Technological Stack 34

4.2.5 Operating Environment 35

4.2.6 Design and Implementation Constraints 35

4.2.7 User Documentation 36

4.2.8 General Constraints 36

4.2.9 Assumptions and Dependencies 36

4.3 External Interface Requirements 36

4.3.1 User Interfaces 36

4.3.2 Hardware Interfaces 37

4.3.3 Software Interfaces 37

4.3.4 Communications Interfaces 37

4.4 Hardware and Software Requirements 37

4.4.1 Hardware Requirements 37

4.4.2 Software Requirements 38

4.5 Functional Requirements 38

4.5.1 Functional Requirement 1 – Recall and Summarization Workflow 38

4.5.2 Functional Requirement 2 - Content Capture 39

4.5.3 Functional Requirement 3 - Timeline Review 39

4.5.4 Functional Requirement 4 - Feedback and Refinement 39

4.6 Other Nonfunctional Requirements 40

4.6.1 Performance Requirements 40

4.6.2 Safety Requirements 40

4.6.3 Security Requirements 40

4.6.4 Software Quality Attributes 40

5 Software Design Document (SDD) 41

5.1 System Architecture 41

5.1.1 Frontend 42

5.1.2 Backend 42

5.1.3 Block Diagram 44

5.2 Use Case Diagram 45

5.3 Activity Diagram 46

5.4 Libraries and Packages Used 47

5.5 Module Description 49

5.5.1 Content Capture Module 49

5.5.2 Summarization & Embedding Module 50

5.5.3 Similarity Search & Retrieval Module 50

5.5.4 Timeline Visualisation Module 51

5.6 Data Flow Diagrams 53

5.6.1 Level 0 DFD 53

5.6.2 Level 1 DFD 53

5.6.3 Level 1.1 DFD 54

5.6.4 Level 1.2 DFD 55

5.6.5 Level 1.3 DFD 56

6 Future Scope **57**

6.1 Intelligent Recall and Multimodal Understanding 57

6.2 Personalization, Reminders, and Knowledge Reinforcement 57

6.3 Cross-Platform Accessibility and Security 58

7 Appendix A - Glossary **59**

8 Appendix B - Acronyms **61**

List of Figures

5.1 High-level Block Diagram 44

5.2 Use Case Diagram 45

5.3 Activity Diagram 46

5.4 Level 0 Data Flow Diagram 53

5.5 Level 1 Data Flow Diagram 54

5.6 Level 1.1 Capture User Activity 54

5.7 Level 1.1.1 Content Capture Module 55

5.8 Level 1.1.2 Summarization and embedding module 55

5.9 Level 1.2 Content Recall 56

5.10 Level 1.2.1 Similarity Search and Retrieval Module 56

5.11 Level 1.3 Timeline Visualization Module 56

List of Tables

2.1 Literature Survey 12

1 Introduction

1.1 Proposed Project

1.1.1 Background

In the modern digital landscape, users continuously consume large volumes of information from diverse online sources such as web pages, research articles, PDFs, and videos. However, recalling specific information or context from previously viewed content becomes difficult over time. Traditional tools like bookmarks or notes only save links or raw text, lacking the ability to understand or organize information based on meaning or relevance. This results in fragmented knowledge and inefficient retrieval when users need to revisit or reuse information.

The proposed AI-Powered Personal Memory System, **RewindYou**, addresses this challenge by automatically capturing, summarizing, and semantically indexing the content users read or watch online. The system leverages natural language processing models for summarization and embedding generation, enabling context-aware storage and intelligent recall. A browser extension captures user sessions, while a cloud-based backend organizes the data into a structured timeline. Through a chat-based interface, users can query their past knowledge in natural language, allowing effortless and meaningful retrieval of previously consumed information.

1.1.2 Objective of the Project

- To develop an AI-powered personal memory assistant that captures and summarizes online content.
- To implement semantic understanding using NLP and embeddings for intelligent recall.
- To enable context-based querying of past web interactions through natural language.
- To ensure user privacy and control, all captured data is stored with explicit user con-

sent, with full options to review, manage, or permanently delete stored sessions at any time.

- To create an intuitive dashboard and timeline for reviewing learned information

2 Literature Survey

2.1 Introduction

The modern internet user encounters vast amounts of information daily, from articles and videos to research papers, yet lacks tools to systematically organize and revisit this content. Existing solutions like bookmarks or read-later apps are limited to URL storage, lacking semantic search capabilities. RewindYou bridges this gap by integrating machine learning with browser technologies to create a dynamic memory assistant. The system operates as a browser extension that captures text from webpages, YouTube transcripts, and PDFs, processes it via summarization and embedding models, and stores it for contextual recall. Users interact through a chat-style interface, asking questions like "What did I learn about transformers last week?" to retrieve AI-generated summaries. By transforming browsing into a searchable knowledge base, RewindYou enhances productivity, learning, and long-term information retention without compromising privacy.

2.2 Literature Survey Table

Table 2.1: Literature Survey

Title	**Authors**	**Key Findings**	**Advantages**	**Disadvantages**	**Inferences**
Web-Summarizer System	Ketan Atri, Mili Adlakha, Shobhit Goel	Uses HTML structure to extract relevant content; avoids summarizing pages with low semantic cohesion.	Reduces noise, supports cross-device readability, uses ERNIE-GEN for summarization.	Poor handling of dynamic/script-heavy content; fixed thresholds may exclude useful info.	Enhances structured summarization but needs flexibility for dynamic content.

Title	Authors	Key Findings	Advantages	Disadvantages	Inferences
Flan-T5 for Abstractive Summarization	Abdul Rahman M. A. Zeyad, Arun Biradar	Performs well on short-text datasets but struggles with multi-doc summaries; ROUGE drops in complex cases.	Context-aware summaries with high BERTScore; tunable and adaptable.	Lacks multilingual support; limited abstractive depth on complex texts.	Effective for simple tasks; less reliable for multi-source or long summaries.
AI Summarization for YouTube	Kanishka Sharma, Sarthak Srivastava, et al.	Pegasus-XSum outperforms BART/T5 in summarization; LLaMA excels in QA tasks; quizzes rated well by users.	Automates summary and quiz creation; modular and scalable design.	Latency and accuracy issues in real-time or niche topics.	Strong for educational use, but needs real-time and domain-specific tuning.
Survey of Vector Databases	Xingrui Xie, Han Liu, et al.	Vector DBs handle unstructured high-dim data; use cosine similarity for retrieval.	High-speed search across image/text; scalable and suitable for multimodal AI.	Setup is complex; tools may lack GPU/analytics support.	Excellent for AI tasks, but needs infra and cost consideration.
Semantic Search via Human Knowledge	Zachary Weinfeld, Lubomir Stanchev	Verb substitution yields best MAP; WASER outperforms TF-IDF, BM25.	Improves semantic accuracy using WordNet; compatible with various embeddings.	Limited to verbs; performance varies across datasets.	Combines linguistic strategies with transformers effectively.
Web Content Extraction	Azin Ekramifard, Hoda Mashayekhi, et al.	99% F-score using Random Forest and novel structural/semantic features.	Generalizes across layouts; maintains content coherence.	Complex feature set (126); requires labeled data.	High performance, but demands compute and data labeling.

Title	Authors	Key Findings	Advantages	Disadvantages	Inferences
FAISS for Sentence Similarity	Premanand Ghadekar, Prajwal Patil, et al.	FAISS enables fast search with clustering; BERT embeddings give better performance.	High-speed, scalable for large semantic datasets.	Sensitive to tuning and embedding quality.	Good for chatbot/search apps, tuning critical.
SBERT for Text Matching	Xiaohui Liu, Fumin Chen, et al.	SBERT-Inter improves Spearman correlation; sentence-level features outperform token-level.	Fine-grained similarity capture using multi-view fusion.	Tested only on Chinese STS-B; expensive computation.	Precise semantic similarity, but needs broader validation.
AI-based Student Assessment	Bambang Dwi Wijanarko, Bachtiar, et al.	98% sensitivity but low specificity; normalization helps interpretation but not statistical accuracy.	Scalable essay grading, reduces human bias.	Misclassifies nuanced answers; low multilingual ability.	Best for structured evaluation, lacks nuance handling.

2.3 Detailed Literature Survey

2.3.1 A Pragmatic Approach to Web-Summarizer System[1]

Authors: Ketan Atri, Mili Adlakha, Shobhit Goel

This paper introduces a web content summarization framework designed to combat information overload by integrating content extraction, evaluation, and abstractive summarization into a unified system. It improves upon existing browser-based tools by refining the preprocessing phase, ensuring that only coherent and meaningful content is passed to the summarization model.

Methodology:

- The system architecture includes three main modules:
  - Content Extractor: Removes ads, scripts, and irrelevant sections while preserving essential HTML structure and tags.
  - Content Evaluator: Assesses extracted text using metrics like content length ratio, unique named entity dominance, and text cohesion to determine suitability for summarization.
  - Summarization Module: Uses abstractive summarization models such as ERNIE-GEN or ProphetNet to generate human-like summaries.
- The extraction process involves data cleaning, visual componentization through Degree of Coherence (DoC) for grouping semantically related sections, and contextual enhancement using parent and neighboring HTML tags.
- Evaluation ensures that only high-quality, coherent content is summarized, improving final output clarity.

Advantages:

- Effectively reduces web page noise by filtering out non-informative elements.
- Leverages HTML structure for better semantic understanding and relevance.
- Enhances cross-device readability through concise, structured summaries.
- Integrates advanced abstractive models (ERNIE-GEN, ProphetNet) for more natural language summaries.

Disadvantages:

- Struggles with dynamic or script-heavy content, leading to incomplete extraction.
- Fixed evaluation thresholds may exclude useful but unconventional content.
- Limited adaptability to rapidly changing web layouts or AJAX-based pages.

Inferences:

- The framework improves structured summarization by focusing on content quality before summarization.
- Shows potential for multi-document and dynamic content summarization in future work.
- A flexible, adaptive evaluation mechanism could further enhance robustness and generalization.

2.3.2 Advancements in the Efficacy of Flan-T5 for Abstractive Text Summarization: A Multi-Dataset Evaluation Using ROUGE and BERT Scores[2]

Authors: Abdulrahman Mohsen Ahmed Zeyad, Arun Biradar

This study investigates the performance of Flan-T5 in abstractive text summarization across multiple benchmark datasets such as XSum, CNN/DailyMail, Multi-News, Newsroom, and Gigaword. Using ROUGE and BERTScore for evaluation, Flan-T5 achieved a ROUGE-L score of 0.5021 on the Gigaword dataset, demonstrating strong coherence and contextual accuracy. The model configuration balances output quality and computation. While effective for short, concise summaries, the model's performance declines for multi-document and long-text summarization. The research highlights metric limitations in capturing true semantic quality and emphasizes the need for diverse human-curated datasets and multilingual extensions in future work.

Methodology:

- Configured Flan-T5 for summarization with a maximum sequence length of 512 tokens and beam search width of 4.
- Evaluated across five summarization datasets — XSum, CNN/DailyMail, Multi-News, Newsroom, and Gigaword.
- Used ROUGE and BERTScore metrics to assess performance based on fluency, coherence, and semantic alignment.
- Compared Flan-T5's performance on short-form versus multi-document summarization tasks.

- Analyzed limitations of automatic evaluation metrics and proposed improved dataset diversity for benchmarking.

Advantages:

- Generates context-aware and coherent summaries with high BERTScore.
- Adaptable and tunable, balancing accuracy and computational efficiency.
- Performs particularly well on short-text datasets like Gigaword and XSum.
- Validates Flan-T5's versatility across multiple summarization benchmarks.

Disadvantages:

- Performance drops for multi-document or long-text summarization (e.g., Multi-News).
- Limited multilingual support, restricting generalization across languages.
- Evaluation metrics (ROUGE, BERTScore) insufficiently capture deep semantic nuances.
- Dependent on dataset domain and structure for optimal results.

Inferences:

- Flan-T5 is highly effective for concise, single-document summarization but less robust for complex, multi-source content.
- Emphasizes the need for improved semantic evaluation metrics beyond ROUGE.
- Suggests expanding to multilingual and domain-specific summarization research.
- Reinforces the potential of instruction-tuned models like Flan-T5 for flexible NLP applications.

2.3.3 AI-Powered Transcript Summarization and Question Answering for YouTube and Wikipedia[3]

Authors: Kanishka Sharma, Sarthak Srivastava, Harsh Pilania, Pranjal Kumar Maurya, Ayushi Prakash

This paper introduces a system for automated summarization, question answering, and quiz generation using YouTube transcripts and Wikipedia articles. It combines advanced NLP models Pegasus-XSum for summarization, Llama for contextual QA, and spaCy for named entity recognition to process content efficiently. The system emphasizes a modular, scalable architecture suitable for educational and knowledge management applications, providing concise summaries, accurate answers, and interactive quizzes.

Methodology:

- Data Acquisition: Collects transcripts via the YouTube Transcript API or web scraping Wikipedia pages.
- Preprocessing: Cleans and normalizes text for downstream tasks.
- Summarization: Uses Pegasus-XSum to generate concise abstracts.
- Question Answering: Employs vector embeddings stored in Pinecone for retrieval and Llama for generating context-aware answers.
- Quiz Generation: Uses named entity recognition and distractor generation techniques to create fill-in-the-blank and multiple-choice quizzes.
- Evaluation: Benchmarked on datasets achieving ROUGE-1 score of 46.7 for summarization and F1 score of 78.4 for QA.

Advantages:

- Automates summary, QA, and quiz creation in a single pipeline.
- Modular and scalable design, allowing easy updates and extensions.
- Outperforms baseline models: Pegasus-XSum ¿ BART/T5 for summarization, LLaMA excels in QA.
- Supports educational and knowledge management applications effectively.

Disadvantages:

- Latency issues in real-time processing.
- Distractor generation accuracy can be inconsistent.
- Performance may degrade on niche or domain-specific topics.

Inferences:

- Demonstrates the potential of integrated NLP pipelines for education and content summarization.
- Highlights the need for real-time optimization and domain-specific fine-tuning.
- Modular architecture allows for future expansions, such as improved quiz generation and multilingual support.

2.3.4 A Brief Survey of Vector Databases[4]

Authors: Xingrui Xie, Han Liu, Wenzhe Hou, Hongbin Huang

This survey presents an extensive overview of vector databases, which efficiently manage and query high-dimensional data such as text and images through vector embeddings. These databases enable semantic similarity search using metrics like Euclidean distance or cosine similarity. The paper analyzes core indexing and querying algorithms—K-Means, Locality-Sensitive Hashing (LSH), Hierarchical Navigable Small Worlds (HNSW), and Product Quantization (PQ)—and compares leading vector database systems, including Pinecone, Chroma, and Milvus, in terms of scalability, integration, and performance.

Methodology:

- Indexing Process: Vectors are transformed and compressed for storage efficiency.
- Querying Mechanism: Similarity-based search using rough and detailed comparisons via cosine or Euclidean distance.
- Algorithms Discussed:
  - K-Means for clustering similar vectors.
  - LSH for bucket-based approximate search.

  - HNSW for graph-based nearest-neighbor retrieval.
  - PQ for dimensionality reduction and memory optimization.

- Comparative Evaluation:
  - Pinecone: Best suited for real-time cloud applications.
  - Chroma: Provides tight integration with LLMs for AI workflows.
  - Milvus: Offers framework compatibility (e.g., PyTorch, TensorFlow) and open-source scalability.

Advantages:

- Enables high-speed search and retrieval across unstructured, multimodal data.
- Scalable and efficient, supporting massive datasets.
- Ideal for cross-modal retrieval tasks involving text, image, and audio data.

Disadvantages:

- Complex setup and infrastructure requirements, especially for large-scale deployments.
- Some databases lack GPU acceleration or built-in analytics tools.
- High costs associated with maintaining real-time or cloud-hosted systems

Inferences:

- Vector databases are central to modern AI systems, enabling semantic search and retrieval across modalities.
- Continued research should focus on algorithm optimization, latency reduction, and cross-modal embedding alignment.

2.3.5 Enhancing Transformer-Based Semantic Search by Integrating Human-Crafted Knowledge into Sentence Embeddings[5]

Authors: Zachary Weinfeld, Lubomir Stanchev

This paper introduces WASER (Weighted Average of Synthetic Embeddings Representation), a novel method designed to enhance semantic search by infusing WordNet-based human knowledge into transformer embeddings. Instead of relying solely on raw text data, WASER creates synthetic text variations by replacing verbs with semantically similar alternatives from WordNet, then combines these embeddings through weighted averaging. This enriched representation improves semantic understanding and retrieval accuracy across transformer models.

Methodology:

- Synthetic Variation Generation: Verbs in the original text are replaced with semantically similar alternatives using WordNet, producing multiple text variations.
- Embedding and Aggregation: Each variation is embedded using a transformer model; embeddings are combined via weighted averaging to form a more semantically rich representation.
- Evaluation: Conducted on the Cranfield benchmark using 23 transformer models, achieving a 2% increase in Mean Average Precision (MAP), with optimal performance at n=4 variations.
- Analysis: Explores POS-based impacts, identifying verb substitution as the most effective category for improving retrieval.
- Trade-offs: Notes that hyperparameter tuning affects performance and computational efficiency.

Advantages:

- Enhances semantic accuracy by incorporating structured lexical knowledge.
- Outperforms traditional methods like TF-IDF and BM25.
- Model-agnostic approach, compatible with multiple transformer architectures.
- Reduces word sensitivity, maintaining meaning despite surface-level text changes.

Disadvantages:

- Limited to verb-based augmentation, potentially missing improvements from other POS categories.

- Performance varies across datasets due to linguistic differences.
- Increased computational cost from generating and embedding multiple text variations.

Inferences:

- WASER demonstrates that combining linguistic knowledge with transformer models yields measurable gains in retrieval performance.
- The approach validates the benefit of integrating structured resources like WordNet into modern embedding workflows.
- Future work could extend WASER to multi-POS substitutions and explore dynamic weighting mechanisms for broader semantic enrichment.

2.3.6 Automatic Webpage Content Extraction Based on Structural and Semantic Features[6]

Authors: Azin Ekramifard, Hoda Mashayekhi, Marziea Rahimi

This paper proposes a machine learning-based approach for web content extraction by leveraging both structural and semantic features. HTML pages are segmented into blocks, which are analyzed using features from the DOM tree and textual content. A random forest classifier is trained to distinguish useful content from boilerplate. Evaluated on a dataset of 868 manually labeled webpages, the method achieves an F-score of 99%, outperforming traditional rule-based tools like Traffiatura and Justext. The study demonstrates the robustness and scalability of ML-based content extraction compared to static rule-based methods.

Methodology:

- HTML Segmentation: Webpages are divided into blocks based on the DOM structure.
- Feature Extraction: Each block is represented using a combination of structural (DOM relationships, block hierarchy) and semantic (cosine similarity, content density) features.
- Model Training: A random forest classifier is trained on a labeled dataset of 868 webpages.

- Evaluation: Performance compared against rule-based methods, achieving F-score of 99%.
- Feature Innovations: Introduces seven novel features including block relationships and inter-block similarity to enhance classification accuracy.

Advantages:

- High accuracy (F-score 99%), outperforming rule-based extraction tools.
- Generalizes across diverse webpage layouts while maintaining content coherence.
- Scalable and robust, reducing the need for frequent manual rule updates.
- Effectively combines structural and semantic features for reliable content identification.

Disadvantages:

- Requires complex feature engineering (126 features in total).
- Depends on manually labeled training data for supervised learning.
- Computationally intensive, especially for large-scale web crawling.

Inferences:

- Demonstrates that ML-based extraction surpasses rule-based approaches in accuracy and adaptability.
- Highlights the importance of feature-rich representations combining structure and semantics.
- Suggests future work could explore deep learning methods for automated feature learning and handling dynamic or script-heavy pages.

2.3.7 Sentence Meaning Similarity Detector Using FAISS[7]

Authors: Premanand Ghadekar, Sahil Mohite, Omkar More, Prajwal Patil, Sayantika, Shubham Mangrule

This study investigates the use of FAISS (Facebook AI Similarity Search) for efficient semantic search in high-dimensional vector spaces. Sentences are encoded into BERT embeddings, which are indexed using FAISS to enable fast retrieval of semantically similar sentences. On a dataset of 14,500 sentences, search time decreased from 16 seconds to 2 seconds, demonstrating FAISS's capability for large-scale, real-time similarity searches in NLP applications such as chatbots and question-answering systems.

Methodology:

- Embedding Generation: Sentences are converted into numerical vectors using BERT embeddings.
- Indexing with FAISS: Vectors are indexed to allow fast nearest-neighbor retrieval in high-dimensional space.
- Similarity Measurement: Cosine similarity is used to evaluate the closeness of vectors.
- Performance Evaluation: Search time and retrieval accuracy are assessed on a dataset of 14,500 sentences.
- Analysis: Explores advantages of FAISS, including clustering-based speedup, and highlights limitations such as sensitivity to embedding quality.

Advantages:

- Enables high-speed search across large semantic datasets.
- Scalable and suitable for real-time applications like chatbots and search engines.
- Improved retrieval performance using BERT embeddings for semantic understanding.
- Supports clustering and approximate nearest-neighbor search for efficiency.

Disadvantages:

- Performance depends on embedding quality, poor embeddings reduce accuracy.
- Requires careful tuning of FAISS parameters for optimal speed and accuracy.
- Approximate search may miss exact matches in some cases.

Inferences:

- FAISS provides a practical, high-performance solution for semantic search in NLP tasks.
- Integrating vector embeddings with FAISS enables scalable real-time retrieval.
- Future work could focus on embedding optimization and hybrid retrieval strategies to enhance accuracy and robustness.

2.3.8 Research on Sentence Embeddings for Text Matching through Multi-view Interactive Features[8]

Authors: Xiaohui Liu, Fumin Chen, Yong Hu, Xinjian Li

This paper presents SBERT-Inter, an enhanced sentence embedding method designed for text matching by integrating multi-view interactive features. Unlike traditional approaches that either lose sentence interactions (representation-based) or fail to generate reusable embeddings (interaction-based), SBERT-Inter combines token-level interactions with sentence-level semantic differences and similarities. Using a Siamese BERT network and multi-view fusion, the model captures fine-grained semantic relationships, achieving improved performance on benchmark datasets.

Methodology:

- Utilizes a Siamese BERT network to generate sentence embeddings.
- Incorporates multi-view fusion combining:
  - Token self-attention,
  - Embedding differences,
  - Similarity vectors at the sentence level.
- Evaluates performance on the STS-B dataset, measuring Spearman correlation.
- Compares SBERT-Inter against traditional representation- and interaction-based methods.
- Analyzes computational efficiency and adaptability for multi-label text matching tasks.

Advantages:

- Achieves 6% improvement in Spearman correlation, outperforming existing methods.
- Captures fine-grained semantic similarity through multi-view fusion.
- Produces reusable embeddings, enabling efficient downstream applications.
- Applicable to retrieval-augmented systems and personalized knowledge management.

Disadvantages:

- Tested primarily on Chinese STS-B, limiting generalizability.
- Computationally expensive due to token-level interactions and multi-view fusion.
- Requires further evaluation on diverse languages and domains.

Inferences:

- SBERT-Inter effectively balances token-level and sentence-level features for precise semantic similarity.
- Demonstrates potential for text matching and retrieval applications in multilingual or multi-label contexts.
- Future work should focus on broader validation and optimization for computational efficiency.

2.3.9 AI-based Feature Extraction and Cosine Similarity for Automation of Student Learning Assessment [9]

Authors: Bambang Dwi Wijanarko, Bachtiar, Rosilah Binti Hassan, Dina Fitria Murad, Ridho Bramulya Ihsan, Yaya Heryadi, Engkos Achmad Kuncoro, Eka Julianti

This research proposes an automated essay scoring system using NLP and AI-based similarity measurement. The system evaluates student answers by comparing them to key answers using BERT-based embeddings, dimensionality reduction, and cosine similarity. It demonstrates high sensitivity (98.21%), effectively identifying correct responses,

and provides a scalable, objective, and efficient tool for educational assessment, while highlighting challenges in detecting incorrect or nuanced answers.

Methodology:

- Uses BERT-based models: "indo-sentence-bert-base" for Indonesian and "stsb-roberta-base" for cross-lingual semantic similarity.
- Feature Extraction: Generates sentence embeddings from student answers and key answers.
- Dimensionality Reduction: Applies PCA to embeddings to reduce complexity.
- Similarity Measurement: Calculates cosine similarity between student responses and answer keys.
- Standardization Thresholding: Converts scores using z-score normalization to set evaluation thresholds.
- Evaluation: Measures performance via sensitivity and specificity, and uses ANOVA to analyze normalization effects.

Advantages:

- High sensitivity (98.21%), accurately identifying correct answers.
- Scalable and objective, reducing human grading bias.
- Normalization ensures consistency across different responses.
- Integrates AI-driven evaluation with human assessment for efficiency.

Disadvantages:

- Low specificity (50%), leading to misclassification of incorrect or nuanced answers.
- Limited multilingual capability, restricting generalization across languages.
- Relies on structured answer formats for optimal performance.

Inferences:

- Effective for structured essay assessment, providing scalable automation in educational settings.
- Highlights the need for enhancing specificity and handling nuanced or context-rich answers.
- Future work could focus on multilingual adaptation and robust semantic evaluation for diverse datasets.

2.4 Future Scope from Literature

The reviewed literature highlights significant progress in AI-powered summarization, semantic search, web content extraction, and educational assessment systems. Key advances are driven by transformer-based architectures, multi-view embeddings, vector databases, and integration of structured knowledge resources. Future research directions include development of robust and multilingual abstractive summarization models capable of handling multi-document and long-text scenarios, creation of dynamic content extraction and GUI-aware frameworks for script-heavy webpages, and personalized, domain-specific WebAgents for applications in healthcare, education, and finance. There is a need for trustworthy, privacy-preserving retrieval and semantic search systems with optimized embeddings and scalable indexing, as well as cross-modal and generative AI integration to enhance context-aware understanding. Further directions involve adaptive evaluation metrics capturing semantic quality, interactive AI-driven educational tools with real-time scoring and quiz generation, efficient multi-view and multi-modal embedding architectures, and explainable AI frameworks that justify model outputs and agent decisions in high-stakes applications. Collectively, these efforts aim to advance the accuracy, generalization, and practical utility of AI systems in diverse real-world environments.

3 Problem Statement and Solution

3.1 Problem Statement

3.1.1 Problem Statement

With the growing amount of information consumed online through web pages, PDFs, and videos, users often struggle to recall or organize what they have learned. Existing tools like bookmarks or note-taking applications fail to capture the context and meaning of the content, storing only links or plain text without understanding intent. This makes retrieving specific knowledge time-consuming and inefficient. There is a need for an intelligent system that can comprehend, summarize, and recall online content in a context-aware manner, helping users remember what they have read or watched with ease and precision.

3.1.2 Proposed Solution

This project develops an AI-powered Personal Memory System for Web Content Recall that captures, processes, and intelligently retrieves information consumed by users online. The system operates through a browser extension and a cloud-based backend, integrating NLP summarization, semantic embeddings, and contextual recall to enable intelligent knowledge retrieval.

- **Content Capture Agent:** Extracts readable text and metadata from web pages, PDFs, and YouTube transcripts while maintaining source details like title, URL, and timestamp.
- **Summarization Engine:** Uses language models (e.g., FLAN-T5) to generate concise summaries that preserve key ideas and context from the captured content.
- **Embedding Generator:** Converts processed content into vector representations using transformer-based models for semantic similarity and contextual search.

- **Memory Organizer:** Categorizes and stores each captured entry with session data in a cloud database, ensuring secure and structured knowledge storage.
- **Query and Recall Agent:** Allows users to ask natural language questions through a chat interface, retrieving relevant content and summaries based on meaning rather than keywords.
- **Timeline Dashboard:** Displays a chronological view of captured sessions, enabling users to visually navigate and revisit their learning history.

By combining summarization, semantic search, and user-intent-driven recall, the system provides a context-aware digital memory that enhances productivity, learning, and information retention.

4 Software Requirements Specification (SRS)

4.1 Introduction

4.1.1 Purpose

The purpose of RewindYou is to act as a personal AI-powered digital memory assistant that helps users recall and query information they have consumed online. Traditional bookmarks or browser histories store only URLs, which do not help users remember what they actually read, watched, or studied. RewindYou solves this by:

- Capturing webpages, PDFs, and YouTube transcripts.
- Cleaning the content to remove clutter.
- Summarizing the main ideas.
- Storing semantic embeddings in a vector database.
- Enabling users to ask questions and retrieve contextual answers.

4.1.2 Document Conventions

- Sections: Computer Modern, Bold 14.4pt
- Subsections: Computer Modern, Bold 12pt

Bullets or bold fonts are used to identify special information about the application. The rest of the content is written in Computer Modern with a font size of 10pt.

4.1.3 Intended Audience and Reading Suggestions

This document is used for explaining the functional and non-functional requirements of AI-Powered Personal Memory System for Web Content Recall. The document also provides the reader a general understanding about the features and functionalities of the software application.

Types of audience:

- Developers: To implement the system's architecture, AI models, and integration workflows.
- Students & Learners: To use RewindYou for content recall, revision, and quick reference from their uploaded materials.
- Academic and Research Communities: To explore advanced AI applications in information retrieval, summarization, and memory enhancement.

4.1.4 Project Scope

RewindYou is an AI-powered system designed to support content recall and summarization. It tackles common challenges in revision such as retaining important concepts from lengthy materials, connecting related topics, and retrieving information quickly when needed. The platform allows users to upload study resources including textbooks, notes, or research papers, and then generates concise summaries and structured outputs that strengthen memory and recall. Through interactive navigation and iterative review, RewindYou provides a flexible yet organized way to revisit material across academic, research, and self-learning contexts, making the process of recalling and reviewing information more effective.

4.1.5 Overview of Developer's Responsibilities

The developers are responsible for:

1. Implement browser extension to capture user-consumed content.
2. Design content cleaning pipelines to strip irrelevant elements.
3. Integrate summarization and embedding models via HuggingFace.

4. Implement ChromaDB + FAISS vector store for semantic memory.
5. Build APIs (FastAPI/Flask) for search, timeline, and chat.
6. Develop React + Tailwind UI for querying and reviewing past knowledge.
7. Ensure compliance with privacy and ethical data handling.

4.2 Overall Description

4.2.1 Product Perspective

RewindYou is a modular platform designed to assist in automating content recall and structured summarization. It integrates advanced language models with information retrieval and organization techniques to generate coherent topic overviews and memory-focused outputs. The system is not intended to replace active learning but rather to complement it by providing intelligent aids for reviewing materials and adaptive workflows that guide users through efficient recall sessions.

4.2.2 Product Functions

- Capture and organize content from different study or reference materials.
- Provide concise summaries to make recall faster and easier.
- Enable quick retrieval of the most relevant information when questions are asked.
- Offer chat-style Q&A for interactive recall and understanding.
- Present daily timeline summaries for ongoing review.
- Allow users to delete, exclude, or tag memories for better personalization and control.

4.2.3 User Classes and Characteristics

1. General Users (Students/Professionals):
    - Require easy setup, minimal configuration.
    - Use for learning recall or productivity.

2. Advanced Users/Researchers:
   - May store large volumes of content.
   - Need precise semantic search..
3. System Developers:
   - Maintain models, backend APIs, and DB.

4.2.4 Technological Stack

Operating System: Windows, Linux, macOS

Programming Languages: Python 3.10+, Javascript

Integrated Development Environment (IDE): Visual Studio Code

AI Frameworks:

- **FLAN-T5 (Fine-Tuned Language-Net Text-to-Text Transfer Transformer):** FLAN-T5 is a fine-tuned version of Google's T5 model that converts every NLP task into a text-to-text format. It has been instruction-tuned on diverse datasets to follow natural language prompts effectively, enabling it to perform tasks like summarization, reasoning, question answering, and text generation with high generalization. Its versatility and lightweight inference make it ideal for downstream applications requiring contextual understanding and structured generation.
- **All-MiniLM-L6-v2 (Sentence Transformer for Semantic Embeddings):** All-MiniLM-L6-v2 is a compact yet high-performing model from Sentence Transformers optimized for generating dense semantic embeddings. It maps sentences or paragraphs to fixed-size vectors, capturing their meaning for tasks like semantic search, clustering, and similarity detection. Despite having only 6 transformer layers, it achieves strong performance while maintaining low latency and memory usage making it well-suited for large-scale or real-time applications such as retrieval-augmented generation and contextual recall systems.
- **FAISS (Facebook AI Similarity Search):** FAISS is an open-source library developed by Meta AI for efficient similarity search and clustering of high-dimensional vectors. It enables rapid retrieval of nearest neighbors from large embedding datasets using optimized indexing structures and GPU acceleration. By combining quantization

techniques and vector compression, FAISS supports scalable semantic search, recommendation systems, and retrieval-augmented generation pipelines with minimal latency and memory overhead.

Backend: Flask/FastAPI with REST APIs - The backend is implemented using lightweight Python frameworks such as Flask or FastAPI, enabling scalable and high-performance API management. These frameworks facilitate seamless integration with AI models via REST endpoints, handling inference requests, and coordinating communication between the frontend and the database.

Database: MongoDB and ChromaDB - MongoDB is a NoSQL document-oriented database designed for flexibility and scalability. It efficiently stores unstructured and semi-structured data such as user activity logs, session histories, and AI-generated content, enabling dynamic schema evolution. ChromaDB, on the other hand, serves as a specialized vector database for managing and querying high-dimensional embeddings. It powers semantic search, context retrieval, and similarity-based reasoning in LLM-powered systems by enabling fast and accurate vector lookups.

Frontend: React.js - The frontend uses React.js to build dynamic, component-based user interfaces that update in real-time based on backend data. Its modular architecture ensures maintainability and flexibility for integrating AI-powered features.

Version Control System: GitHub - GitHub manages version control, collaboration, and deployment workflows. Branching and pull requests enable safe feature integration, while CI/CD pipelines can be automated with GitHub Actions to ensure reliability, traceability, and smooth team collaboration throughout development.

4.2.5 Operating Environment

- Python 3.10+ runtime environment.
- Supported on Linux, Windows, and MacOS platforms.
- Browsers: Chromium
- Storage: MongoDB for memory and ChromaDB for vectors.

4.2.6 Design and Implementation Constraints

- Must use free/open-source tools only.

- Summarization models limited by local compute resources.
- Browser API restrictions (no direct OS-level access).

4.2.7 User Documentation

- A README file with setup and installation instructions.
- Usage documentation for developers.
- Example workflows and tutorials to guide end users.

4.2.8 General Constraints

- System performance depends on network connectivity and website response times.
- API usage costs and rate limits may restrict throughput and scalability.

4.2.9 Assumptions and Dependencies

- Assumes users explicitly allow content capture.
- Assumes HuggingFace models can run locally.
- Vector database backends and storage systems are properly maintained.

4.3 External Interface Requirements

4.3.1 User Interfaces

The interface shall be intuitive, engaging, and accessible, providing tools to capture and recall content from what users read or watch. Users should be able to:

- Interactively explore recalled concepts through chat-style Q&A.
- Review daily timeline summaries and detailed topic overviews for effective revision.

The interface shall support both desktop and mobile environments with adaptive layouts and interactive visualization tools for exploring recalled content.

4.3.2 Hardware Interfaces

- Users can access the system through standard desktops, laptops, and smartphones.
- Monitor screen - results will be displayed via web interface.
- Keyboard - used for entering queries.
- Mouse - used for web dashboard interaction.

4.3.3 Software Interfaces

- Parsing: BeautifulSoup for HTML document parsing.
- Storage: MongoDB for storing memory.
- Vector Database: ChromaDB for vector embeddings.
- AI Models: FLAN T5, all Mini LM l6 v2 and FAISS.

4.3.4 Communications Interfaces

The system will rely on secure communication protocols:

- HTTPS to encrypt all data transfers between clients and servers.
- Token-based authentication and session management to ensure account security and personalized recall sessions.

4.4 Hardware and Software Requirements

4.4.1 Hardware Requirements

Server side:

- Minimum: 8GB RAM, Dual-core CPU.
- Recommended: 16GB RAM, Quad-core CPU, and GPU acceleration for faster embeddings.

Client side:

- Monitor screen - to display summaries and timeline.
- Keyboard - to input queries.
- Mouse - to navigate web dashboard usage.

4.4.2 Software Requirements

Server side:

- Python 3.10 or above as the primary development environment.
- ChromaDB for vector storage.
- FLAN T5 for summarization and all Mini LM l6 v2 for vector embeddings

Client side:

- Any OS capable of running a modern web browser that supports JavaScript and HTML5.
- A browser extension for capturing and recalling content directly while reading or watching.

4.5 Functional Requirements

4.5.1 Functional Requirement 1 – Recall and Summarization Workflow

This feature allows the system to extract, organize, and present key information from content the user reads or watches. This feature is of High priority.

1. There should be a user-friendly interface to input topics, queries, or areas of focus for recall.
2. The system should provide structured outputs that maintain logical flow and topic coherence.

3. The system should automatically capture relevant content from user interactions for recall purposes.
4. Summaries should maintain consistency across related topics and sessions.
5. The system should store and track versions of summaries and recall sessions for future reference.

4.5.2 Functional Requirement 2 - Content Capture

This feature enables the system to capture content from user interactions with text, videos, or other supported sources while ensuring full control over what is recorded.

- The system should extract only the content explicitly selected or permitted by the user for recall.
- Captured content should be organized for easy retrieval and review.
- Strict privacy controls should prevent unauthorized capture of sensitive or unrelated content, ensuring users maintain full ownership and control over their data.

4.5.3 Functional Requirement 3 - Timeline Review

This feature provides chronological or structured summaries for revisiting content over time.

- The system should generate daily or session-based timeline summaries for ongoing review.
- Users should be able to navigate through timelines to track and revisit previously captured content.

4.5.4 Functional Requirement 4 - Feedback and Refinement

This feature supports iterative improvement of recall outputs based on user input.

- Users should be able to provide feedback on summary accuracy, relevance, and completeness.
- The system should adapt future summaries and recall outputs based on user feedback.

4.6 Other Nonfunctional Requirements

4.6.1 Performance Requirements

The system should be capable of handling multiple recall and summarization tasks simultaneously without significant latency. It should scale across topics, lengths of content, and user queries, ensuring responsiveness in both interactive sessions.

4.6.2 Safety Requirements

The system should ensure that all recalled content is accurate, trustworthy, and contextually appropriate. It should detect and flag conflicting or ambiguous information to prevent misunderstandings. Measures should be implemented to avoid exposing sensitive or private information during recall sessions. Additionally, the system should provide warnings or guidance when the user requests recall from incomplete or potentially misleading sources.

4.6.3 Security Requirements

- User data, including captured content, summaries, and personalization settings, should be encrypted both in transit and at rest.
- Access control must ensure that only authorized users can retrieve, view, or modify their captured content and recall outputs.
- The system should allow users to control what content is captured, ensuring no unauthorized recording of unrelated material.

4.6.4 Software Quality Attributes

- Portability – The system should support multiple platforms including web browsers, desktops, mobile devices, and browser extensions.
- Availability – The service should remain accessible at all times, with backup and fail over mechanisms to ensure data integrity during outages.
- Reliability – The system should maintain accurate and coherent recall outputs even under high-load conditions.

5 Software Design Document (SDD)

5.1 System Architecture

The RewindYou system integrates intelligent frontend and backend modules to function as a personal digital memory assistant. The Frontend comprises a browser extension and a web dashboard that capture and display user interactions. The extension extracts content from webpages, YouTube videos, and PDFs using JavaScript APIs, while the React.js + Tailwind CSS dashboard allows users to view summaries, search memories, and ask queries through a chat interface. The Backend handles all processing and reasoning tasks. It cleans captured text, summarizes it using FLAN-T5, and generates semantic embeddings via All-MiniLM-L6-v2. These embeddings are stored in ChromaDB, which uses FAISS internally for fast similarity search. When users query the system, the backend performs semantic retrieval to return the most relevant information. Together, these components create a seamless and scalable architecture that captures, organizes, and retrieves digital content intelligently, enabling users to recall past information naturally and efficiently.

5.1.1 Frontend

The Frontend Module of RewindYou provides an intuitive and user-friendly interface for interacting with the system. It serves as the bridge between the user and the intelligent backend, enabling seamless content capture, query interaction, and memory review. The frontend is designed for accessibility, clarity, and efficiency, ensuring that users can easily manage and explore their stored information. Key features include:

- **Browser Extension:** Built using JavaScript and Chrome Manifest V3 APIs, the extension captures various types of online content, including:
  - Webpage text content extracted via DOM parsing and Readability.js.
  - YouTube video transcripts obtained through the YouTube Data API.
  - PDF document text processed using pdf.js.

  It also provides simple controls such as Start/Stop Recording, Exclude Page, and Settings for customizable user interactions.
- **Web Dashboard:** Developed using React.js and Tailwind CSS, the dashboard enables users to explore captured content through multiple views and tools.
  - Chat Interface: Allows users to ask natural language questions such as "What did I read about transformers last week?" and receive contextual summaries.
  - Timeline View: Displays daily summaries of captured data organized by date and source type.
  - Search and Filter: Enables filtering by tags, content type (e.g., PDF, Webpage, YouTube), and keywords for quick retrieval.
  - Content Management: Provides options to edit, delete, or exclude specific stored memories to maintain a clean dataset.
- **Input / Output:** Input: User actions such as initiating content capture, submitting queries, or browsing the timeline.
  Output: Summaries, conversational responses, and organized visualizations of stored content.

5.1.2 Backend

The Backend Module forms the intelligent processing core of RewindYou. It manages the heavy computational tasks, including text cleaning, summarization, embedding generation, and semantic retrieval. This module ensures that captured data is transformed into

structured, searchable, and meaningful digital memories that users can access efficiently through the frontend interface.

- **Content Processing:** Preprocesses captured text by removing advertisements, boilerplate sections, and irrelevant data using Readability.js and custom filtering rules. This ensures the model receives clean and meaningful input.
- **Summarization Service:** Utilizes the FLAN-T5 model from Hugging Face Transformers for abstractive summarization. The service converts lengthy or unstructured text into concise summaries, optimizing both storage and recall. It is hosted as an independent microservice within the backend architecture.
- **Semantic Embedding Generator:** Employs the All-MiniLM-L6-v2 model from SentenceTransformers to convert processed and summarized text into 384-dimensional dense vector embeddings. These embeddings represent the semantic meaning of content, enabling similarity-based recall.
- **Vector Database (Memory Store):** Implements ChromaDB as the primary vector database, internally powered by FAISS (IndexFlatL2) for high-speed similarity search. It stores both embeddings and their associated metadata (title, URL, timestamp, and content type), supporting efficient Top-K nearest neighbor retrieval using cosine similarity.
- **Backend API Layer:** The communication interface between the frontend and backend, built using FastAPI or Flask, handles user requests, content submission, semantic search queries, and retrieval of summarized results.
- **Input / Output:** Input: Cleaned text content received from the frontend and user queries submitted through the chat interface.
  Output: Generated embeddings, top-K semantic matches, and final context-aware responses.

5.1.3 Block Diagram

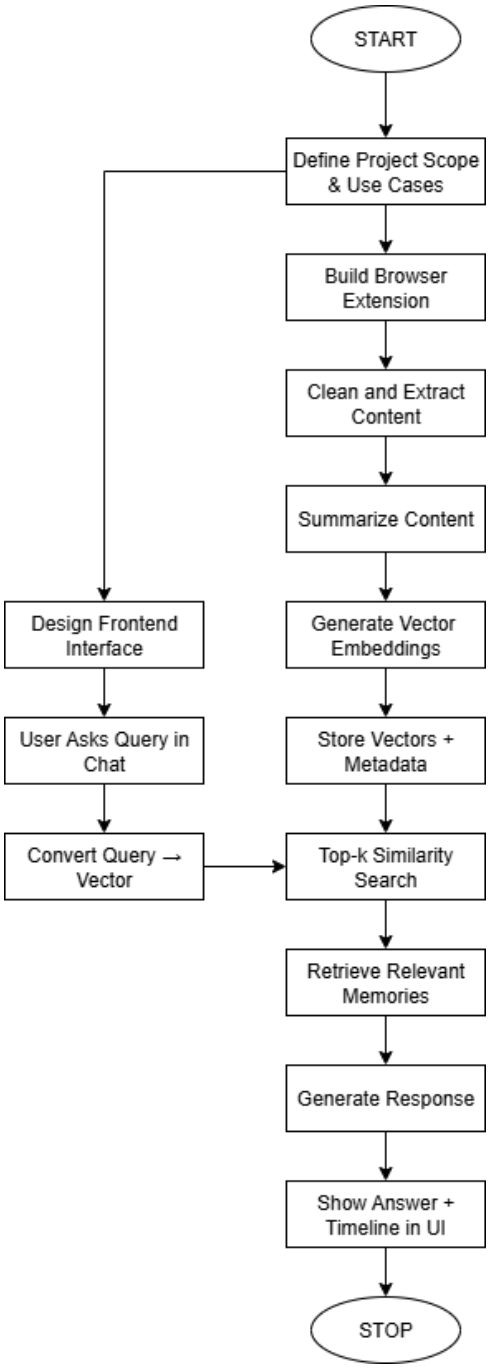

Figure 5.1: High-level Block Diagram

5.2 Use Case Diagram

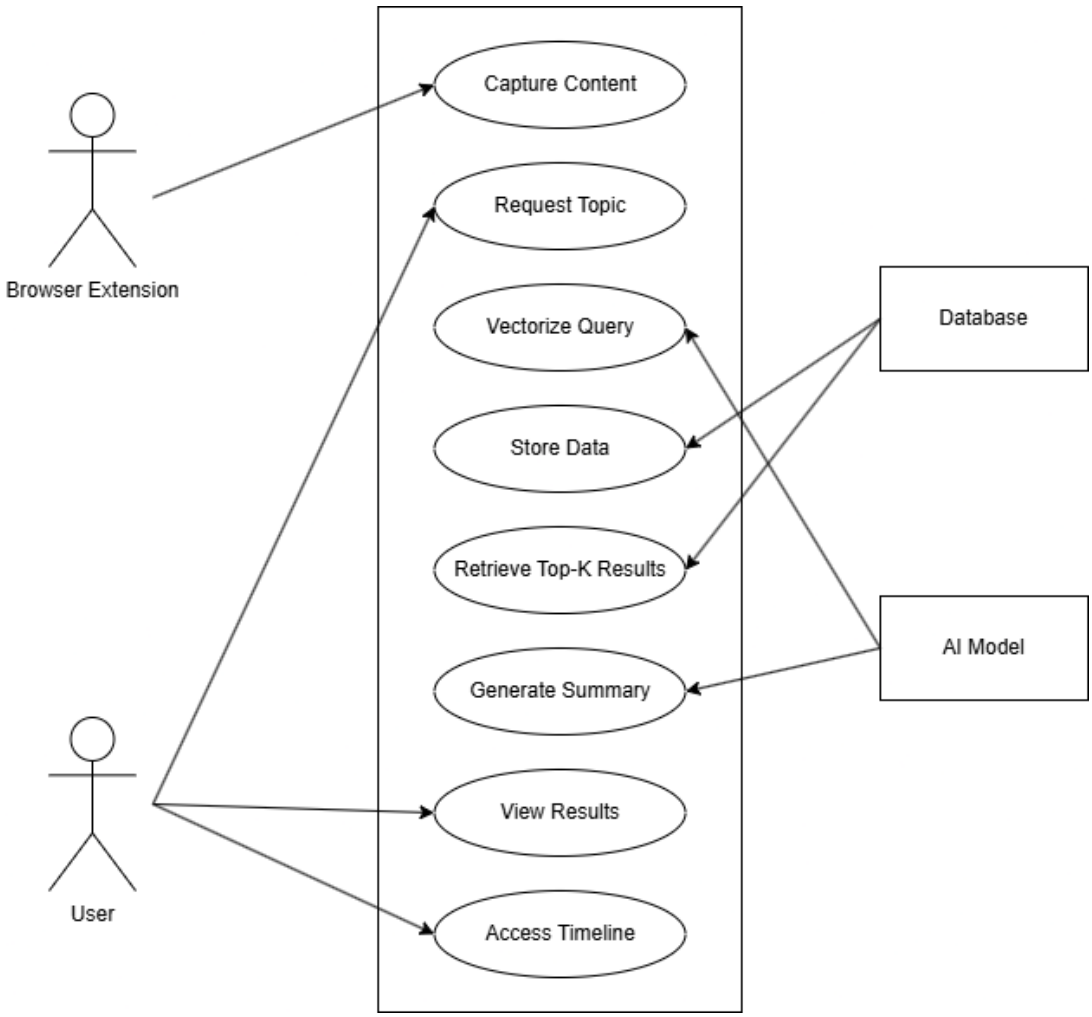

Figure 5.2: Use Case Diagram

The Use Case Diagram outlines the interaction between the user and the Web Navigation Agent. The main actor, the User, performs activities such as logging in, uploading resumes, and submitting job URLs. The system responds by validating inputs, processing navigation, performing semantic matching, and automating job applications. This diagram emphasizes system functionality from the user's perspective and shows how each use case contributes to achieving the system's objectives.

5.3 Activity Diagram

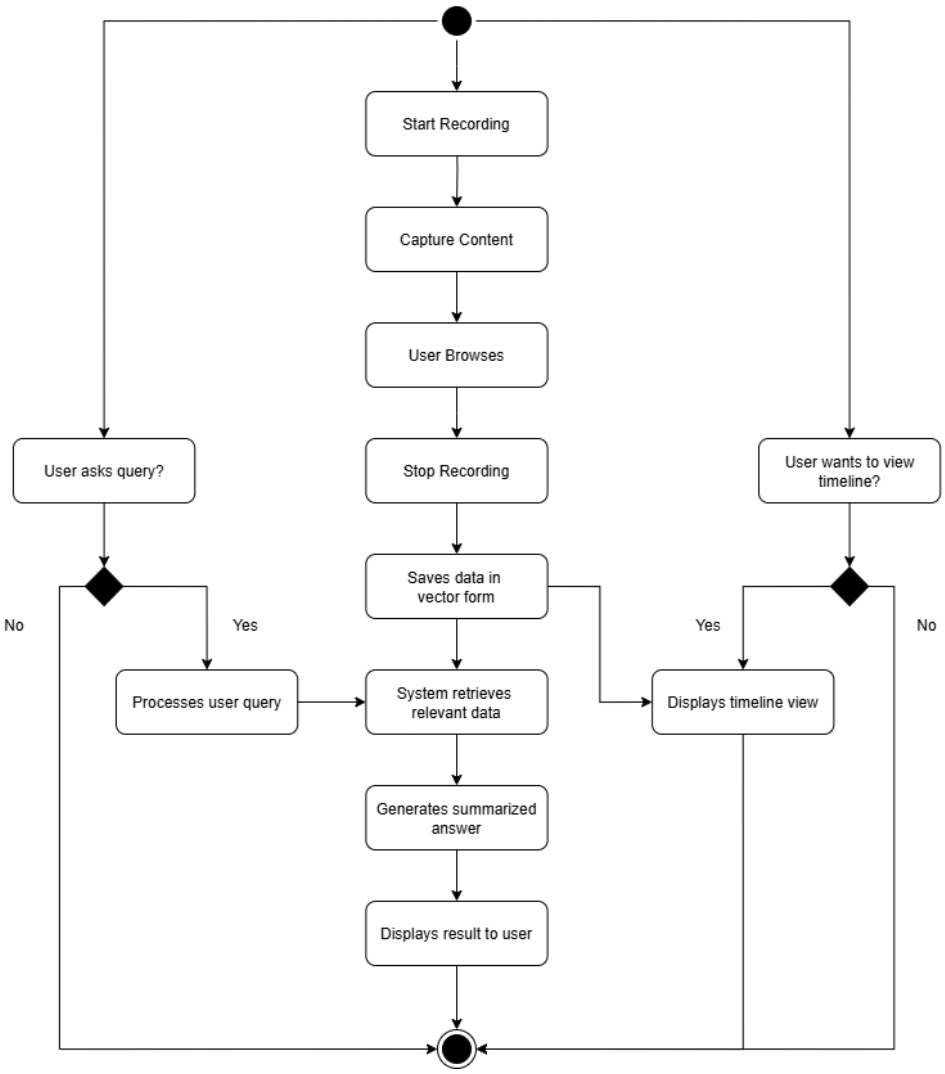

Figure 5.3: Activity Diagram

The Activity Diagram depicts the complete workflow of the automated job application process. It starts with user authentication and input collection, followed by backend actions such as webpage navigation, DOM parsing, semantic matching, and form submission. Decision points and feedback loops are shown to represent dynamic control flow, ensuring that errors are handled efficiently and that the agent can adapt its behavior in real time.

5.4 Libraries and Packages Used

The system is implemented in Python 3.10+ and makes use of the following key libraries and packages. Each library plays a specific role in enabling content capture, summarization and vector embeddings.

Frontend Frameworks

- **React:** Used for building the interactive web dashboard that allows users to view, search, and query their captured content seamlessly.
- **Tailwind CSS:** Provides a utility-first CSS framework for designing a responsive and visually consistent user interface with minimal effort.

Backend Frameworks

- **Flask:** Serves as the lightweight backend framework for managing API routes, handling user requests, and connecting the frontend with AI processing modules.

Language Models and Embeddings

- **Hugging Face Transformers (FLAN-T5):** Used for abstractive summarization and natural language understanding, converting lengthy text into concise, meaningful summaries.
- **Sentence Transformers (All-MiniLM-L6-v2):** Generates dense semantic embeddings for captured text, enabling efficient similarity matching and contextual recall.

Vector Databases and Retrieval

- **ChromaDB:** Manages storage and retrieval of text embeddings with metadata, supporting semantic search and contextual memory recall.
- **FAISS:** Provides high-speed similarity search and vector indexing, enabling fast retrieval of semantically related embeddings from large datasets.

Databases

- **MongoDB:** A NoSQL document database used to store user profiles, captured content metadata, and system logs with flexible schema design.

These libraries collectively form the technological foundation of the AI-Powered Personal Memory System for Web Content Recall.

5.5 Module Description

5.5.1 Content Capture Module

- **Description:** The Content Capture Module is the system's entry point that captures user selected content web pages, PDFs, or YouTube videos using a browser extension. It extracts text or transcripts along with metadata (URL, title, timestamp) and cleans the content by removing HTML tags, ads, and noise. The cleaned text is sent to the summarization module, while metadata is stored in MongoDB for the timeline module.
- **Inputs:** Webpage URLs, PDF files, or video links provided by the user.
- **Outputs:** Clean, structured textual content ready for summarization.
- **Libraries Used:** Browser APIs (Chrome Extension APIs),, Readability.js, PDF.js, YouTube Transcript API, MongoDB for metadata storage, BeautifulSoup, FastAPI backend communication

Algorithm 1: Content Capture Module

```
Input: Active user tab
Output: cleaned_text, metadata
Initialize browser extension listener;
Detect active user tab and identify content type (HTML, PDF, or video);
if content type is HTML then
    Extract visible text using Readability.js API;
end
else if content type is PDF then
    Extract visible text using PDF.js or similar API;
end
else if content type is video then
    Extract transcript using YouTube Transcript API;
end
metadata ← { "URL", "title", "timestamp" };
Store metadata in MongoDB;
Preprocess extracted content by removing HTML tags, ads, scripts, special
  characters, and extra whitespace;
Validate readability of cleaned text;
Forward cleaned text to Summarization and Embedding Module;
return (cleaned_text, metadata);
```

5.5.2 Summarization & Embedding Module

- **Description:** This module takes preprocessed text, generates a concise summary using FLAN-T5, and converts the summary or user query into semantic embeddings using the Sentence Transformer (all-MiniLM-L6-v2). The embeddings capture the meaning of the text for efficient storage in ChromaDB or similarity-based retrieval.
- **Inputs:** Clean, structured textual content.
- **Outputs:** Vector embeddings
- **Libraries Used:** Flan T5 , all-MiniLM-L6-v2 , FastAPI

Algorithm 2: Summarization and Embedding Module

Input: preprocessed_text
Output: summary, vector_embeddings
Receive preprocessed text from Preprocessing Module;
Summarization using FLAN-T5:;
encoder_output ← Encoder(preprocessed_text) converts input into contextual embeddings;
embeddings ← pass through multiple feed-forward layers in encoder and decoder to capture semantic meaning;
summary ← Decoder(embeddings) generates concise and coherent summary;
Embedding Generation using all-MiniLM-L6-v2:;
tokens ← Tokenize(summary);
token_vectors ← Convert each token into vector representation;
contextualized_vectors ← Process token_vectors through 6 Transformer layers to learn contextual relationships;
vector_embeddings ← Pool contextualized_vectors to obtain dense sentence embedding;
return (summary, vector_embeddings);

5.5.3 Similarity Search & Retrieval Module

- **Description:** This module is responsible for answering user queries by performing semantic search on the stored vector embeddings. It compares the query vector with vectors in ChromaDB using cosine similarity and retrieves the most relevant summaries. The best-matching results are returned as an answer or context snippet to the user.

- **Inputs:** Query Vector, Stored vectors in ChromaDB.
- **Outputs:** Generated textual summary to the user from top k similar elements.
- **Libraries Used:** ChromaDB, FAISS, FastAPI

Algorithm 3: Retrieval and Response Generation Module

Input: query_vector
Output: final_response, metadata
Receive query_vector from user input or memory query;
Perform cosine similarity search in ChromaDB using query_vector;
results ← Retrieve top-k most similar vectors along with their metadata;
Rank results based on contextual relevance and semantic similarity;
selected_result ← Choose the highest-ranked relevant result;
Summarize and format selected_result into a coherent, readable response;
return (final_response, metadata);

5.5.4 Timeline Visualisation Module

- **Description:** The Timeline Module provides a visual interface for users to explore their captured content chronologically. It retrieves metadata from MongoDB (which stores timestamps, content types, and titles) and displays it in an interactive timeline view. Users can filter by date and click on timeline items to revisit summaries or query results
- **Inputs:** User-selected timeline range
- **Outputs:** Chronologically ordered content items(title, type, timestamp, link), Visualization-ready data for the React Frontend.
- **Libraries Used:** MongoDB (metadata + session storage), React.js, Tailwind CSS for frontend.

Algorithm 4: Timeline Generation Module

Input: timeline_filter_request

Output: formatted_timeline_data

Receive timeline_filter_request from frontend;

Fetch corresponding metadata entries from MongoDB based on filter criteria;

Sort retrieved entries chronologically in descending order (most recent first);

timeline_data ← Structure entries into timeline-friendly format (date → content_summary);

Send formatted timeline_data to frontend for visualization and rendering;

return formatted_timeline_data;

5.6 Data Flow Diagrams

This section presents the data flow diagrams for the Job Application Automation System. It includes Level 0 and Level 1 Data Flow Diagrams (DFDs), as well as Use Case, Activity, and Class diagrams to capture the system's structure, behavior, and data interactions.

5.6.1 Level 0 DFD

The Level 0 Data Flow Diagram represents the overall architecture of the RewindYou system. It shows how the user interacts with the main system components — Content Capture, Summarization and Embedding, Similarity Search, and Timeline Visualization. The user initiates the process by providing URLs, documents, or queries through the frontend. These inputs flow to the backend, where the system captures, processes, stores, and retrieves memories. The summarized outputs, search results, or timeline views are then returned to the user, illustrating the complete system boundary and primary data exchanges.

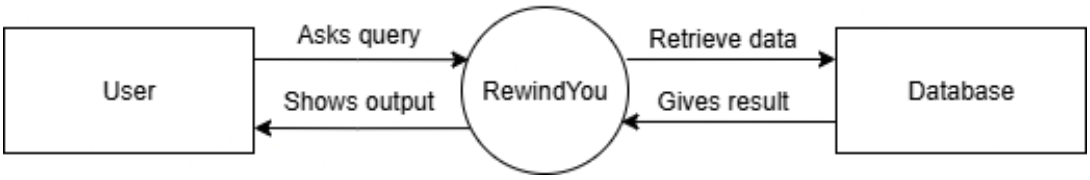

Figure 5.4: Level 0 Data Flow Diagram

5.6.2 Level 1 DFD

The Level 1 DFD breaks down the high-level RewindYou system into its core processes: Capture User Activity, Content Recall, and Timeline Visualization. Each module interacts with databases and AI models to handle specific functionalities. Captured content is processed into summaries and embeddings, stored in the vector database for retrieval, or visualized in timelines. This level provides a detailed view of how user data flows across modules and backend services.

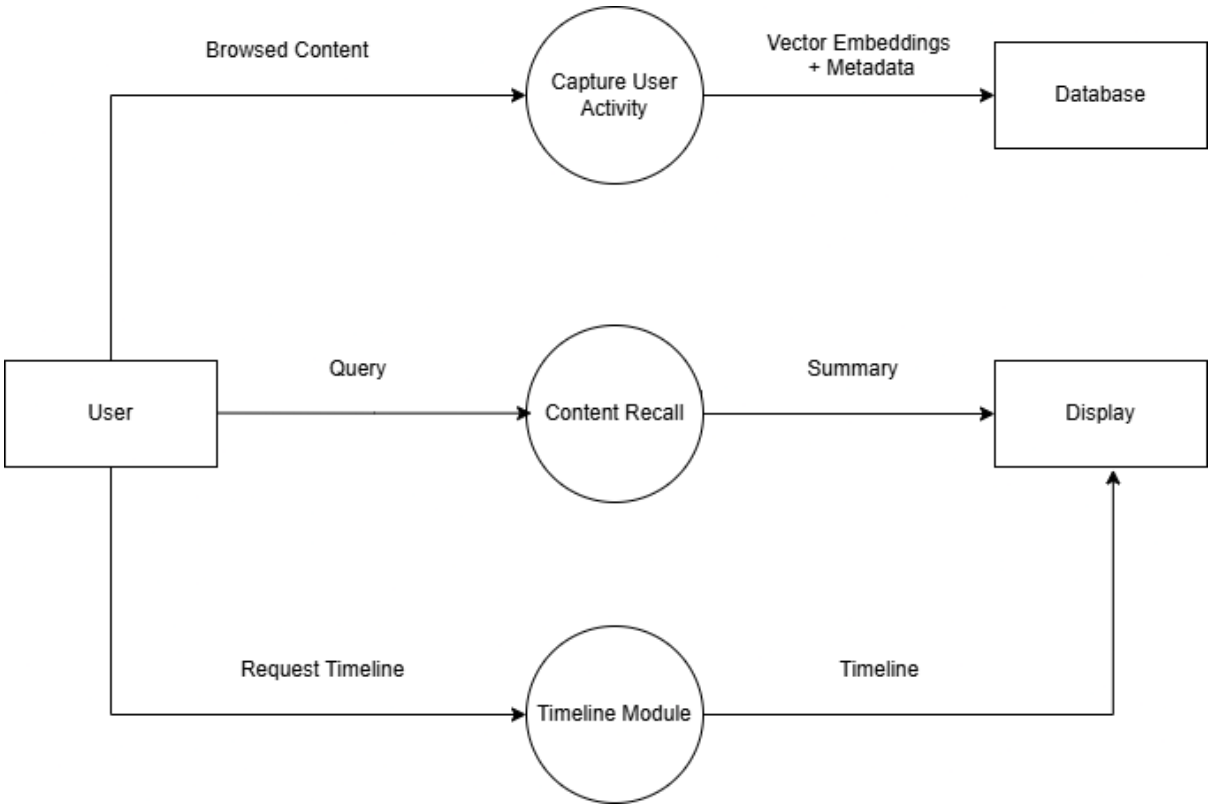

Figure 5.5: Level 1 Data Flow Diagram

5.6.3 Level 1.1 DFD

The Level 1.1 DFD focuses on the Capture User Activity process. It shows how content from different sources — web pages, PDFs, and YouTube videos — is detected and extracted in real time using the browser extension. The extracted content is then cleaned, summarized, embedded, and stored. This diagram highlights the interaction between the user, browser extension, preprocessing, and storage layers during content capture.

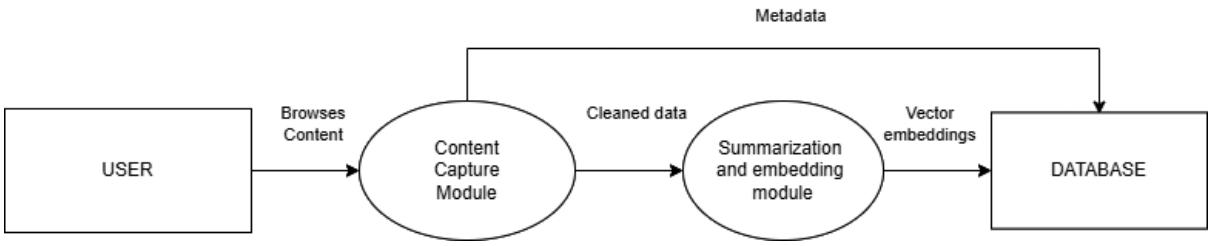

Figure 5.6: Level 1.1 Capture User Activity

Level 1.1.1 DFD

The Level 1.1.1 DFD depicts the internal data flow within the Content Capture Module. It starts with the user selecting a source (web page, PDF, or video). The browser extension extracts the visible text or transcript and metadata such as URL, title, and timestamp. The raw text is cleaned to remove HTML tags and noise, producing structured, readable content that is sent to the Summarization Module. Metadata is simultaneously stored in MongoDB for later use in timeline visualization.

Figure 5.7: Level 1.1.1 Content Capture Module

Level 1.1.2 DFD

The Level 1.1.2 DFD illustrates the Summarization and Embedding Module. The preprocessed text is passed to the FLAN-T5 model, which generates concise summaries. These summaries are converted into semantic vector embeddings using Sentence Transformer (all-MiniLM-L6-v2). The resulting embeddings and related metadata are stored in ChromaDB for future similarity-based retrieval, enabling intelligent recall and search capabilities.

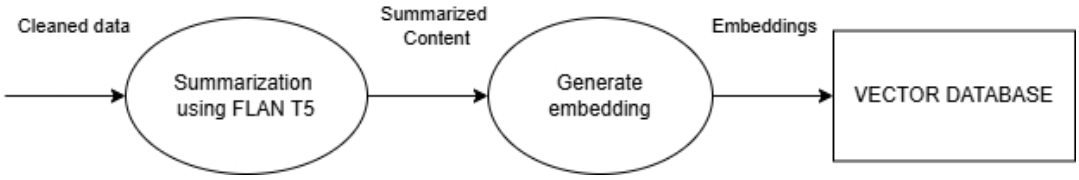

Figure 5.8: Level 1.1.2 Summarization and embedding module

5.6.4 Level 1.2 DFD

The Level 1.2 DFD focuses on the Content Recall process. When a user submits a query, it is converted into a vector and compared against stored embeddings in ChromaDB using cosine similarity. The system retrieves the most relevant matches, summarizes them, and sends readable results back to the frontend. This diagram highlights the intelligent retrieval mechanism of RewindYou's memory system.

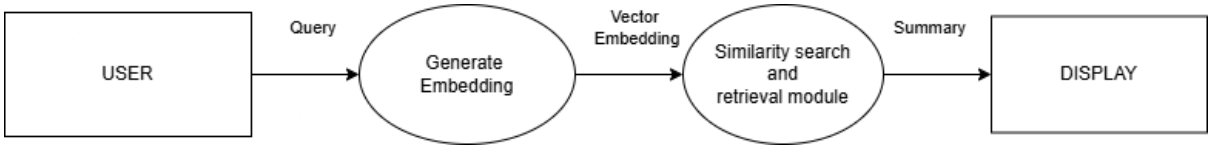

Figure 5.9: Level 1.2 Content Recall

Level 1.2.1 DFD

The Level 1.2.1 DFD represents the internal flow of the Similarity Search and Retrieval Module. The query vector is matched with stored embeddings, the top-k results are fetched and ranked, and their summaries are formatted into coherent responses. These responses, along with relevant metadata, are returned to the user interface, completing the recall process.

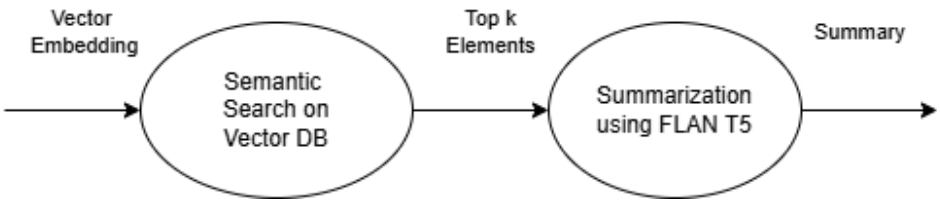

Figure 5.10: Level 1.2.1 Similarity Search and Retrieval Module

5.6.5 Level 1.3 DFD

The Level 1.3 DFD focuses on the Timeline Visualization Module. It retrieves metadata (title, content type, timestamp) from MongoDB based on user-selected date ranges. The data is sorted chronologically and structured into a visualization-ready format. The frontend displays this as an interactive timeline, allowing users to explore and revisit captured memories seamlessly.

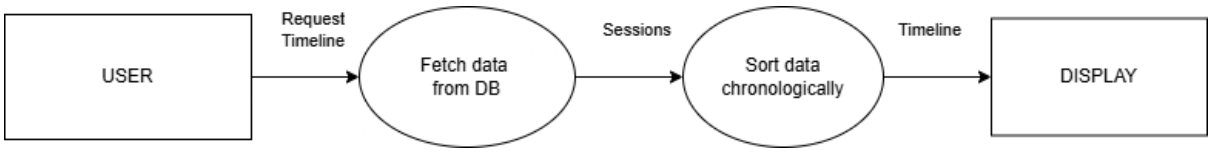

Figure 5.11: Level 1.3 Timeline Visualization Module

Together, these sublevel DFDs demonstrate how theRewindYou system processes and transforms information from raw web data collection and DOM parsing to semantic embedding, retrieval, and intelligent decision-making.

6 Future Scope

The RewindYou - AI-Powered Personal Memory System for Web Content Recall holds strong potential for future enhancement and expansion, especially with the integration of advanced multimodal processing, personalization, and cross-platform capabilities. The following directions outline opportunities to further improve system intelligence, accessibility, and user experience:

6.1 Intelligent Recall and Multimodal Understanding

- **Voice-Based Memory Queries:** Integrate speech recognition models (such as Whisper) to allow users to recall information using natural voice commands, enabling hands-free and intuitive interaction with their digital memory.
- **Optical Character Recognition (OCR) for Scanned Documents:** Implement OCR functionality using frameworks like Tesseract or PaddleOCR to extract text from scanned PDFs and images, expanding the system's capability to capture offline or image-based learning materials.
- **Visual Context Linking:** Develop mechanisms to connect images, diagrams, and text within captured sessions, enabling multimodal recall and supporting users who rely on visual memory cues for learning.

6.2 Personalization, Reminders, and Knowledge Reinforcement

- **Contextual Reminders and Spaced Repetition:** Introduce intelligent reminders based on user activity and learning frequency, where the system periodically resurfaces important topics or summaries for long-term retention, inspired by cognitive reinforcement techniques.
- **Adaptive Memory Weighting:** Implement a dynamic prioritization mechanism that learns which topics or sources are most relevant to the user and adjusts recall results

accordingly for more personalized responses.

- **Sentiment and Focus Analysis:** Incorporate lightweight NLP models to identify user interest or engagement level within captured content, enabling tailored recommendations and topic clustering.
- **Personal Learning Insights:** Provide analytics such as most-viewed topics, knowledge growth trends, and daily learning summaries to help users understand and track their information consumption patterns.

6.3 Cross-Platform Accessibility and Security

- **Mobile Application and Cross-Device Sync:** Extend the system to mobile platforms and ensure seamless synchronization across devices via secure cloud storage, allowing users to access their digital memory anywhere.
- **Offline Access and Caching:** Enable temporary offline retrieval of recent memories or summaries, improving usability in low-connectivity environments while maintaining security.
- **API Integration for External Platforms:** Develop APIs to integrate with productivity tools (e.g., Notion, Obsidian, or Google Drive), enabling unified access and intelligent recall across multiple knowledge sources.

7 Appendix A - Glossary

LLM (Large Language Model) – An advanced AI model capable of understanding and generating human-like text, used for semantic analysis and task reasoning.

DOM (Document Object Model) – A structured representation of a web page used to identify, extract, and interact with HTML elements.

Embedding – A vector representation of textual or visual data that captures semantic meaning for similarity matching.

Semantic Matching – A process that compares embeddings to find conceptually similar content.

Vector Database – A specialized database designed to store and query high-dimensional embeddings efficiently.

FAISS (Facebook AI Similarity Search) – A library for efficient similarity search and clustering of dense vectors, used for semantic search.

NLP (Natural Language Processing) – A subfield of AI focused on enabling machines to understand and interpret human language.

API (Application Programming Interface) – A communication interface allowing integration between modules.

User Query – A request or question entered by the user, typically in natural language, to retrieve specific information or content from a system.

Semantic Search – A search mechanism that retrieves results based on contextual meaning rather than exact keyword matches.

ChromaDB - A vector database designed to store and efficiently retrieve embeddings for fast semantic search and similarity-based queries.

Content Capture – The process of collecting relevant text or elements from web pages

using DOM parsing and NLP methods.

Browser Extension – A small software add-on that enhances or customizes a web browser's functionality and user experience.

FLAN-T5 - A transformer-based language model fine-tuned on instruction datasets to perform a wide range of natural language tasks like summarization, translation, and question answering.

All-MiniLM-L6-v2 - A compact sentence embedding model that converts text into numerical vectors for semantic search, clustering, and similarity comparison.

Timeline - A visual or structured representation of events or activities arranged in chronological order.

Content Recall - The process of retrieving previously captured or stored information in response to a user's query.

8 Appendix B - Acronyms

AI - Artificial Intelligence

LLM - Large Language Model

NLP - Natural Language Processing

API - Application Programming Interface

HTML - Hypertext Markup Language

CSS - Cascading Style Sheets

DOM - Document Object Model

JSON - JavaScript Object Notation

FAISS - Facebook AI Similarity Search

UI - User Interface

GUI - Graphical User Interface

DBMS - Database Management System

HTTPS - Hypertext Transfer Protocol Secure

IDE - Integrated Development Environment

HTTP - Hypertext Transfer Protocol

GPU - Graphics Processing Unit

OCR - Optical Character Recognition

FLAN-T5 - Fine-tuned Language Net - Text-To-Text Transfer Transformer

Bibliography

[1] K. Atri, M. Adlakha and S. Goel, *A Pragmatic Approach to Web-Summarizer System,* 2021 International Conference on Intelligent Technologies (CONIT), Hubli, India, 2021, pp. 1-5, doi: 10.1109/CONIT51480.2021.9498320.

[2] A. M. Ahmed Zeyad and A. Biradar, *Advancements in the Efficacy of Flan-T5 for Abstractive Text Summarization: A Multi-Dataset Evaluation Using ROUGE and BERTScore,* 2024 International Conference on Advancements in Power, Communication and Intelligent Systems (APCI), KANNUR, India, 2024, pp. 1-5, doi: 10.1109/APCI61480.2024.10616418.

[3] K. Sharma, S. Srivastava, H. Pilania, P. K. Maurya and A. Prakash, *AI-Powered Transcript Summarization and Question Answering for YouTube and Wikipedia,* 2025 2nd International Conference on Computational Intelligence, Communication Technology and Networking (CICTN), Ghaziabad, India, 2025, pp. 777-782, doi: 10.1109/CICTN64563.2025.10932550.

[4] X. Xie, H. Liu, W. Hou and H. Huang, *A Brief Survey of Vector Databases,* 2023 9th International Conference on Big Data and Information Analytics (BigDIA), Haikou, China, 2023, pp. 364-371, doi: 10.1109/BigDIA60676.2023.10429609.

[5] Z. Weinfeld and L. Stanchev,*Enhancing Transformer-Based Semantic Search by Integrating Human-Crafted Knowledge Into Sentence Embeddings,* 2025 19th International Conference on Semantic Computing (ICSC), Laguna Hills, CA, USA, 2025, pp. 118-121, doi: 10.1109/ICSC64641.2025.00022.

[6] A. Ekramifard, H. Mashayekhi and M. Rahimi, *Automatic Webpage Content Extraction Based on Structural and Semantic Features,* 2025 11th International Conference on Web Research (ICWR), Tehran, Iran, Islamic Republic of, 2025, pp. 276-280, doi: 10.1109/ICWR65219.2025.11006249.

[7] P. P. Ghadekar, S. Mohite, O. More, P. Patil, Sayantika and S. Mangrule, *Sentence Meaning Similarity Detector Using FAISS,* 2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA), Pune, India, 2023, pp. 1-6, doi: 10.1109/ICCUBEA58933.2023.10392009.

[8] X. Liu, F. Chen, Y. Hu and X. Li, *Research on Sentence Embeddings for Text Matching through Multiview Interactive Features,* 2023 3rd International Conference on Neural Networks, Information and Communication Engineering (NNICE), Guangzhou, China, 2023, pp. 43-47, doi: 10.1109/NNICE58320.2023.10105673.

[9] B. D. Wijanarko et al., *AI-Based Feature Extraction and Cosine Similarity for Automation of Student Learning Assessment,* 2023 24th International Arab Conference on Information Technology (ACIT), Ajman, United Arab Emirates, 2023, pp. 1-6, doi: 10.1109/ACIT58888.2023.10453927.

[10] Python Software Foundation. *Python Documentation.* Online. Available: `https://docs.python.org/3`. Accessed: Oct. 20, 2025.

[11] Flan-T5. *Flan-T5 Documentation.* Online. Available: `https://huggingface.co/docs/transformers/en/model_doc/flan-t5`. Accessed: Oct. 20, 2025.

[12] all-MiniLM-L6-v2. *all-MiniLM-L6-v2 Documentation.* Online. Available: `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`. Accessed: Oct. 20, 2025.

[13] Hugging Face. *Transformers Documentation.* Online. Available: `https://huggingface.co/docs/transformers`. Accessed: Oct. 20, 2025.

[14] FastAPI. *FastAPI Documentation.* Online. Available: `https://fastapi.tiangolo.com`. Accessed: Oct. 20, 2025.

[15] Meta Platforms, Inc. *React Documentation.* Online. Available: `https://react.dev`. Accessed: Oct. 20, 2025.

[16] GitHub, Inc. *GitHub Documentation.* Online. Available: `https://docs.github.com`. Accessed: Oct. 20, 2025.

[17] Stack Overflow. Online. Available: `https://stackoverflow.com`. Accessed: Oct. 20, 2025.